فایل آموزشی ASP.NET

تفکیک‌شده برای دو سطح: Mid-Level و Senior

تهیه و تنظیم : علیرضا بهمنش راد - https://www.linkedin.com/in/behmaneshrad

هدف جلسه

در این جلسه وارد **هسته درونی** **ASP.NET Core** می‌شویم؛ یعنی جایی که فریمورک واقعاً تصمیم می‌گیرد:

- یک Request چگونه وارد سیستم می‌شود
- چگونه پردازش می‌شود
- و چگونه Response تولید می‌شود

این جلسه مرز **برنامه‌نویس فریمورک‌محور** و **توسعه‌دهنده‌ای که فریمورک را می‌فهمد** را مشخص می‌کند.

ساختار زمانی پیشنهادی

- بخش اول (Junior → Mid-level) : **۳۰ دقیقه**
- بخش دوم (Senior-level) : **۳۰ دقیقه**

بخش اول – چیزهایی که اگر بلد نباشی «میدلول نیستی»

(Foundations of ASP.NET Core Internals)

این بخش ترکیبی از:

- Pro ASP.NET Core MVC
- ASP.NET Core in Action
- ASP.NET Core Architecture

---

1 فلسفه طراحی ASP.NET Core (۵ دقیقه)

چرا ASP.NET Core این‌طور طراحی شده؟

- **Cross-platform**
- **High-performance**
- **Minimal core – extensible edges**

اصل کلیدی:

ASP.NET Core یک فریمورک «Host-based» است نه «Web-based»

یعنی:

- وب فقط یکی از سناریوهاست
- معماری از ابتدا برای انعطاف طراحی شده

اگر یک Mid-level هنوز فکر می‌کند ASP.NET Core یعنی MVC Controllerها ❌

---

2 Hosting Model و Program.cs (۷ دقیقه)

اجزای اصلی:

- Host
- Web Host
- Application Startup

مفاهیم حیاتی:

- `IHost`
- Generic Host
- Startup با مدل قدیمی `WebApplicationBuilder`تفاوت

باید بداند:

- چرا Startup حذف شد
- ترتیب ساخته شدن:
  - Configuration
  - Logging
  - Dependency Injection
  - Middleware Pipeline

عدم درک این ترتیب = Debug نکردن حرفه‌ای

---

3 Request Pipeline و Middleware (۱۰ دقیقه)

تعریف Pipeline:

زنجیره‌ای از Delegateها که Request را پردازش می‌کنند

ویژگی‌های کلیدی Middleware:

- ترتیب اجرا مهم‌تر از خود Middleware است
- Short-circuit امکان‌پذیر است
- هر Middleware:
  - قبل از Next
  - بعد از Next

نکته طلایی:

MVC خودش یک Middleware است

اگر Mid-level نداند:

- Authentication
- Authorization
- Routing
- Endpoint Execution

همه Middleware هستند → هنوز Internals را نفهمیده است

---

4 Endpoint Routing (۸ دقیقه)

تفاوت قدیم و جدید:

- Routing دیگر فقط MVC نیست
- Routing مستقل از MVC

مفاهیم:

- Endpoint
- Metadata
- Route Matcher

چرا مهم است؟

- Authorization
- Filters
- Versioning

همه بر پایه Endpoint Metadata کار می‌کنند

بخش دوم – چیزهایی که اگر بلد نباشی «سینیور نیستی»

(Deep Internals & Design-Level Understanding)

برگرفته از:

- Framework Design Guidelines
- Designing Distributed Systems with .NET
- Code That Fits in Your Head
- ASP.NET Core Architecture

---

5 Dependency Injection Internals (۸ دقیقه)

DI در ASP.NET Core:

- Simple by design
- Not a “full-featured container”

Service Lifetimes (واقعی، نه حفظی):

- Singleton → App lifetime
- Scoped → Request lifetime
- Transient → Usage-based

اشتباهات کشنده:

- Inject کردن Scoped در Singleton
- Service Locator disguised as DI

نگاه سینیور:

DI برای loose coupling نیست؛ برای **controlling object graph** است

---

6 Filters vs Middleware (۵ دقیقه)

تفاوت مفهومی:

- Middleware → infrastructure-level
- Filters → MVC execution-level

ترتیب اجرای Filters:

- Authorization
- Resource
- Action
- Exception
- Result

تصمیم طراحی مهم:

آیا این Cross-cutting concern است یا Request concern؟

اگر نتواند فرق را توضیح دهد ⇒ هنوز Senior نیست

---

7 Configuration & Options Pattern (۷ دقیقه)

Configuration Sources:

- JSON
- Environment Variables
- Command Line
- Secret Managers

Options Pattern:

- IOptions
- IOptionsSnapshot
- IOptionsMonitor

نگاه حرفه‌ای:

- Configuration = Runtime behavior
- Options = Boundary Contract

Senior می‌داند:

- چه زمانی Snapshot
- چه زمانی Monitor
- و چرا نباید Config مستقیم Inject شود

---

8 ASP.NET Core به‌عنوان یک Framework (۵ دقیقه)

بر اساس Framework Design Guidelines:

اصول:

- Explicit over implicit
- Small abstractions
- No magic unless justified

نتیجه:

- چرا ASP.NET Core Opinionated نیست
- چرا اجازه سوءاستفاده هم می‌دهد!

Senior واقعی:

فریمورک را سرزنش نمی‌کند، طراحی را تحلیل می‌کند

---

9 Mental Model نهایی (۵ دقیقه)

بر اساس Code That Fits in Your Head

مدل ذهنی پیشنهادی:

1. Host starts
2. Services built
3. Pipeline assembled
4. Request enters
5. Middleware chain
6. Endpoint execution
7. Response flows backward

اگر این مدل ذهنی نباشد:

- Debug سخت
- Performance tuning غیرممکن
- معماری ضعیف

جمع‌بندی نهایی جلسه

اگر شرکت‌کننده بعد از جلسه:

- Pipeline را تصور کند ✅
- Middleware را آگاهانه انتخاب کند ✅
- DI را درست استفاده کند ✅

→ Mid-level واقعی است

اگر بتواند:

- تصمیمات طراحی ASP.NET Core را تحلیل کند
- محدودیت‌ها را بفهمد
- Trade-offها را توضیح دهد

→ Senior واقعی است